Slow-Corner Timing Closure of a Streaming Convolution Engine on sky130: Results and Open Questions

[Your Name] · B.S. Electrical Engineering, Case Western Reserve University · October 2026 · github.com/ydss0857/stream-conv-engine-rtl-to-gds

What I am asking. I implemented a verified 3×3 image-convolution engine on the open-source SkyWater sky130 process. It passes the flow's default sign-off (0 DRC, 0 LVS, hold clean at all corners, setup met at the typical corner), but after seven implementation runs it still misses my 80 MHz target at the slow corner (ss, 100 °C, 1.60 V) by 2.27 ns. I have a diagnosis and a plan (§4–5), but this is my first ASIC flow, and I would value an expert's view on whether my reasoning and next steps are sound. The four questions in §6 are where advice would help most.

1. The design

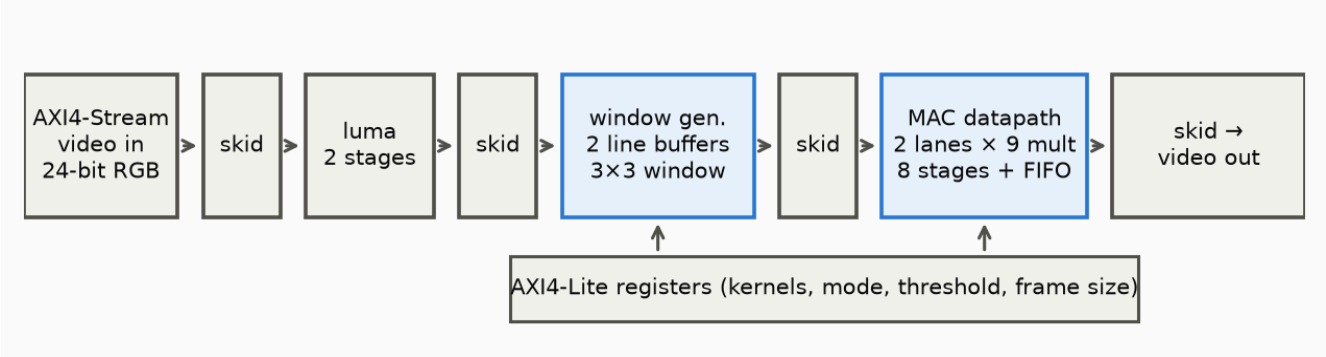


Figure 1. `conv_engine`: AXI4-Stream video in and out, AXI4-Lite control. Two line buffers form 3×3 windows; two programmable 9-multiplier lanes compute blur, sharpen, Laplacian or Sobel magnitude at one pixel per clock.

The RTL is verified against a bit-exact Python model: 32 cocotb [8] tests on two simulators, 1.89 M output pixels compared, formal proofs of the interface protocols (SymbiYosys [9]), and 100% line/branch/expression/toggle coverage. The same RTL meets 150 MHz on a Zynq-7020 FPGA. The 80 MHz ASIC target comes from the application: 720p60 video needs 74.25 MHz.

2. Implementation setup

Item	Setting
Flow / PDK / library	LibreLane 3.0.16 [2] (Yosys [4], OpenROAD [3], Magic [5], KLayout [6], Netgen [7]) · sky130A [1] · `sky130_fd_sc_hd`
Scope	Macro only (no pad ring); line-buffer width 128 pixels (2 × 128 × 8 bits of flip-flops)
Clock / I/O constraints	12.5 ns; input and output delay 20% of the period; minimum input delay raised by 1 ns to model the external driver's clock latency (otherwise every input-to-flop path fails hold at the macro boundary)
Non-default flow settings	`FP_CORE_UTIL` 50; `PNR_CORNERS` = nom_ss_100C_1v60; `EXTRA_EXCLUDED_CELLS` = clkdlybuf4s* (the resizer was using these delay cells as ordinary buffers)
Reset	Single active-low synchronous reset `aresetn` from an input port

3. Results and what has been tried

Metric (baseline run v7)	Result	Metric	Result
Die / std-cell area	0.468 / 0.283 mm²	DRC / LVS / antenna	0 / 0 / 0
Setup @ 12.5 ns, tt / ff	+3.57 / +5.95 ns	Hold, all corners	met
Setup @ 12.5 ns, ss	−2.27 ns, TNS −19 ns	Max-slew violations at ss	≈ 9,200 (worst 2.07 ns vs 0.75 ns limit)

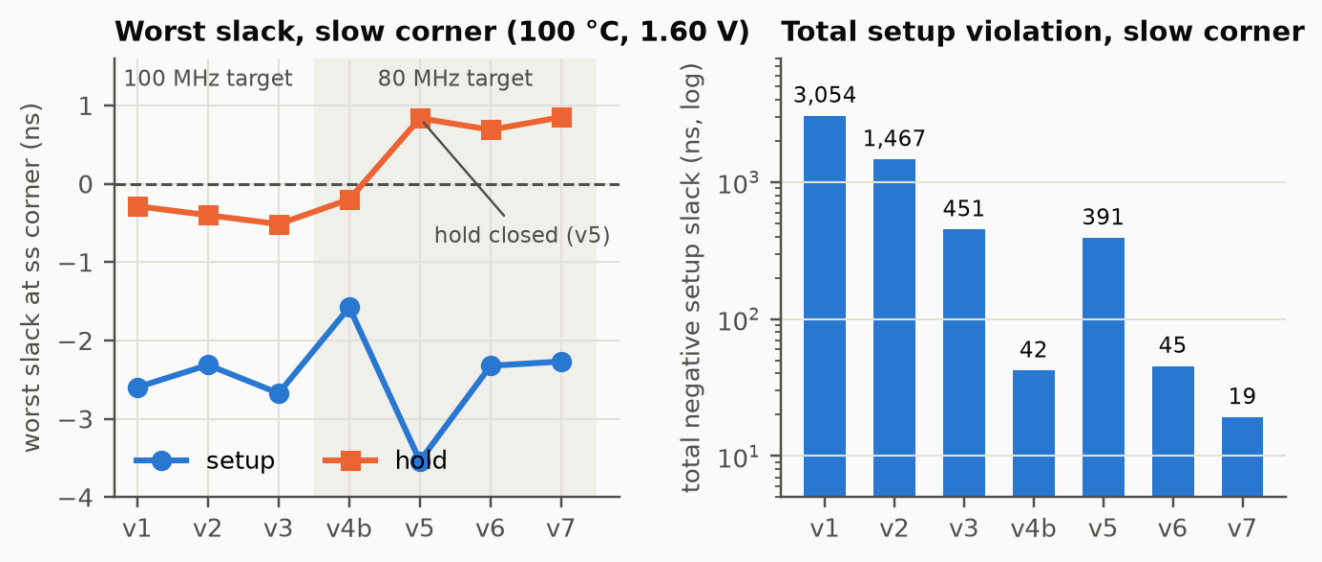


Figure 2. Slow-corner worst slack and total setup violation across seven runs. Runs v1–v3 targeted 100 MHz; v4b onward target 80 MHz.

Run	Change	What the next worst path revealed
v1	First run	A pipeline stall-enable chain ran combinationally through every block
v2	Skid buffers between blocks (RTL)	The resizer used `clkdlybuf4s25` delay cells (≈ 1.4 ns each) as fanout buffers
v3	Excluded those cells	Long chains of `buf_1` sized for the typical corner
v4	Repair margins slew 50% / cap 40%	Repair flagged ~18,000 nets and did not converge in 2 h (abandoned)
v4b	80 MHz target	A ~20-level 12-bit comparator chain in the window generator; hold failing only on input ports
v5	Precomputed counter flags (RTL); realistic min input delay (SDC)	Hold closed; next, a shared clock enable driving ~650 datapath flops
v6	Free-running datapath with credit-based output FIFO (RTL)	Long buffered nets; ~9,000 slew violations at ss
v7	Placement and repair at the ss corner; utilisation 40% → 50%	See §4

4. Current diagnosis

In run v7, 20 paths violate at ss. **14 of them start at the reset input `aresetn`**, which reaches the reset flops through a ~10-stage buffer tree (worst −2.27 ns). The worst path that does *not* start at reset misses by only **−0.61 ns**. Across a line-buffer width sweep, the non-reset slack degrades steadily as the flip-flop line-buffer array grows:

Line-buffer width	64	128	256	512	128, hs library
Violating paths at ss	11	20	58	> 1,000	9
of which from `aresetn`	7	14	0	0	0
Worst non-reset slack (ns)	−0.69	−0.61	−2.83	−4.62	−0.75
Std-cell area (mm²)	0.224	0.283	0.402	0.635	0.378

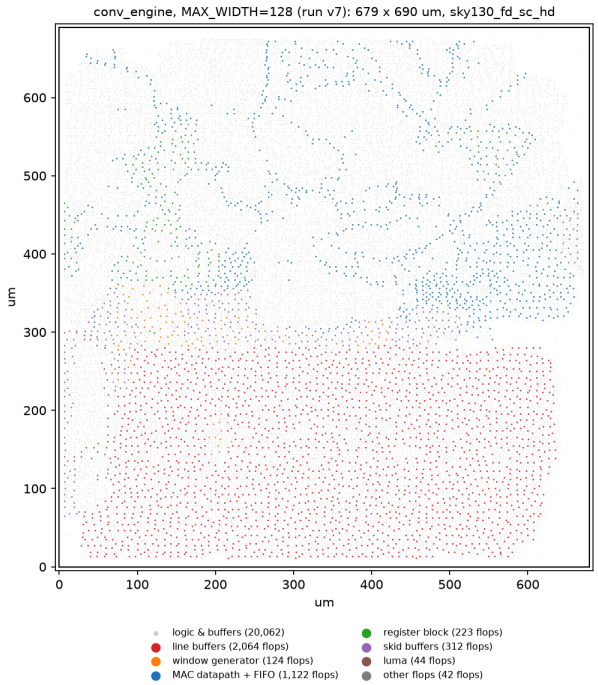


Figure 3. Run v7, every flip-flop coloured by RTL block. The 2,048 line-buffer flops (red) occupy a ~600 × 280 µm region below the window generator (orange).

My reading is that two separate problems overlap:

1. **Reset distribution.** A single synchronous reset from an input port must reach every reset flop (register-block reset values, valid bits, counters, FIFO pointers) within one cycle, after a 2.5 ns input delay. At ss the buffer tree cannot do this.
2. **The flip-flop line-buffer array.** The window generator's read address and enables fan out across the whole array every cycle. I suspect, but have not yet confirmed net by net, that these nets account for most of the ~9,000 slew violations at ss (the worst violators are inserted fanout buffers with ~2.07 ns input transitions). This path dominates as the array grows. Placing and repairing at the ss corner (v7) improved total slack but left the slew-violation count essentially unchanged.

The faster `hs` library closes the logic paths to −0.75 ns (75 MHz, enough for 720p60), at +33% area and 2.2× power.

5. What I plan to try next

1. **Registered, replicated reset:** register `aresetn` at the boundary and give each block its own local reset flop. Also remove resets from flops that don't need one.
2. **Line-buffer read path:** bank the line buffers and register the bank select and decoded address one cycle early, so no read-address net spans the array.
3. **SRAM line buffers** using sky130's OpenRAM-generated [10] `1rw1r` macros. By macro dimensions, two 1 KB macros match the flip-flop cell area at about 440 pixels of width. The PDK only provides a typical-corner model for them, though.
4. **Repair settings in small steps** (slew margin 20 → 30%, wire-length buffering), after v4 showed that large jumps do not converge.

6. Questions

1. **Reset:** for a hardened macro like this, what reset style would you recommend: a registered and replicated synchronous reset, asynchronous assert with synchronous deassert, or treating reset as a

multicycle or false path?

2. **Flip-flop memories:** for a ~2,000-bit flip-flop array read every cycle, is banking with a pipelined read the usual fix, or would you go straight to SRAM (given the typical-only macro model in sky130)?
3. **Slew violations:** why might sizing buffers for the ss corner leave ~9,000 max-slew violations essentially unchanged? Is this a sign that something in my setup (constraints or floorplan) is wrong?
4. **Sign-off target:** LibreLane's default sky130 sign-off checks setup only at the typical corner. For an academic project, is it reasonable to sign off at tt and report the ss Fmax, or should I keep pushing for all-corner closure?

Thank you for reading. The full configuration, constraints, timing reports for every run, and the RTL are in the repository; I am happy to send any of them directly.

Acknowledgment

Development of the RTL, verification environment, flow scripts and this report was assisted by an AI coding assistant (Claude, Anthropic). All design decisions, results and analysis were reviewed by the author, and every reported number comes from the tools and runs recorded in the repository.

References

1. SkyWater Technology and Google, "SkyWater Open Source PDK (sky130)," Apache-2.0. https://github.com/google/skywater-pdk
2. FOSSi Foundation, "LibreLane: an infrastructure for implementing chip design flows," v3.0.16. https://github.com/librelane/librelane
3. T. Ajayi et al., "Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project," in *Proc. 56th Design Automation Conference (DAC)*, 2019.
4. C. Wolf, "Yosys Open SYnthesis Suite." https://yosyshq.net/yosys/
5. J. K. Ousterhout, G. T. Hamachi, R. N. Mayo, W. S. Scott and G. S. Taylor, "Magic: A VLSI Layout System," in *Proc. 21st Design Automation Conference (DAC)*, 1984.
6. M. Köfferlein, "KLayout." https://www.klayout.de
7. R. T. Edwards, "Netgen." http://opencircuitdesign.com/netgen/
8. cocotb contributors, "cocotb: coroutine-based cosimulation testbench environment," v2.1. https://www.cocotb.org
9. YosysHQ, "SymbiYosys (sby): front-end for Yosys-based formal verification flows." https://github.com/YosysHQ/sby
10. M. R. Guthaus, J. E. Stine, S. Ataei, B. Chen, B. Wu and M. Sarwar, "OpenRAM: An Open-Source Memory Compiler," in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2016.